Building context that makes AI outputs smarter - Copilot

Description

Artificial Intelligence is rapidly changing how software is designed, developed, tested, and maintained. GitHub Copilot is one of the leading AI-first development experiences, combining the familiarity of a modern IDE with powerful AI capabilities that can accelerate development and transform the way developers work.

In this hands-on introductory lab, you'll explore the fundamentals of GitHub Copilot and learn how AI-assisted development fits into the modern software engineering workflow. Through guided exercises, you'll discover how to use natural language to generate code, understand existing applications, create tests, refactor implementations, and navigate large codebases more effectively.

The focus of this lab is to demonstrate how applying context at different depths affects the outputs the AI model returns. While artificial intelligence models possess vast general knowledge, their true accuracy and intelligence are unlocked only through precision context engineering. Rather than relying on long, vague prompts that introduce noise, building effective context is about surgically supplying the exact contracts, constraints, state parameters, and reference patterns an AI needs to understand the problem at hand. By the end of the session, you'll understand what is considered context and how we can write/use it to specialise our outputs to be more accurate and precise.

Learning Outcomes

By completing this lab, you will be able to:

- Understand how to avoid "context noise" by passing only relevant information
- Use the 4-layer boundary (Contracts, Target code, Dependencies and Constraints)
- Use surgical # variables (#file:, #symbol, #codebase, etc.)
- Combine reference examples (#file:) with strict boundaries (What should NOT be used)
- Plan before coding and reset chats often to avoid conflicts/context drift
- Automate project conventions using .github/copilot-instructions.md to avoid repeating yourself

Target Audience

This lab is designed for technical leads, delivery managers and technology professionals who want to understand how AI-powered development tools are changing the software delivery lifecycle and how to get started with GitHub Copilot.

Scenario

You need to implement a new calculateOrderTotal function in src/services/orderService.ts.

The function must:
Accept an Order object (src/types/order.ts).
Sum up the item prices multiplied by quantity.
Apply a 10% discount if discountCode equals 'SAVE10'.
Throw a custom error if an item quantity is less than or equal to 0.
Return the calculated numeric total safely without mutating the original order.

Step 1 – The bad context trap

1. Open GitHub Copilot Chat (Ctrl + Cmd + I on macOS / Ctrl + Alt + I on Windows/Linux).
2. Run this poor prompt:
   "Write a function in orderService.ts to calculate order totals with discount code SAVE10. Look at #codebase."
3. Analyze the output:
   Did Copilot reference the deprecated oldDiscountCalculator.ts file?
   Did it use any types or loose equality (==)?
   Did it import the Order type interface properly from order.ts?

Step 2 – Minimal context selection

1. Open a fresh chat thread (Cmd + N or Ctrl + N inside the Chat panel).
2. Identify the exact minimum context needed using the 4-Layer Pyramid:
   Contract Layer: Which file holds the types? (#file:order.ts)
   Target Layer: Which file is being edited? (#file:orderService.ts)
3. Craft a prompt that uses surgical #file: variables to target only those specific items—excluding legacy files or broad workspace context.
4. Goal: Ensure Copilot outputs type-safe code without touching or copying legacy patterns.

Step 3 – Apply the C – R – E – A – T – E Framework

1. Write a structured prompt in Copilot Chat using all 6 elements of C-R-E-A-T-E:
   Context: Tag only #file:order.ts and #file:orderService.ts.
   Role: Senior Backend Engineer focused on immutability.
   Expectations: Write `calculateOrderTotal(order: Order): number`.
   Constraints: Do NOT mutate the `order` parameter. Do NOT use `any`. Do NOT modify #file:order.ts.
2. Test: Throw an Error if `item.quantity <= 0`.
3. Exemplar/Optional: Include a JSDoc example showing input and expected output.

Step 4 – Systematise with rules

1. Review the rules placed in .github/copilot-instructions.md.
2. Open a fresh chat thread. Write a simple prompt:
   "Write a function in #file:orderService.ts to check if an order is eligible for shipping."
3. Verify that Copilot automatically enforces strict typing and immutability without needing explicit instructions in the prompt.